

SEOS Key Store

Content: • Overview ◦ Generic requirements for a full-blown solution ◦ System Architecture • Minimal Keystore ◦ Requirements based on input from Gernot, 2019-02-15 ◦ Threat Model/Assumptions ◦ Design Considerations ◦ Limitations ◦ State ◦ Data structures ▪ data_buffer_t ▪ const_data_buffer_t ▪ app_id_t ▪ key_id_t ◦ Key Store Memory Management ▪ keystore_internal_malloc() ▪ keystore_internal_free() ◦ Key Store Persistence System Connector ▪ keystore_fs_connector_putKeyContent() ▪ keystore_fs_connector_getKeyContent() ▪ keystore_fs_connector_deleteKey() ▪ keystore_fs_connector_wipe() ◦ Key Store Crypto Helper Functions ▪ deriveBlobKey() ▪ keystore_internal_encrypt() ▪ keystore_internal_decrypt() ◦ Keystore exposed API to CryptoServer ▪ Concept ▪ API Properties ▪ keystore_newKey() ▪ keystore_getKey() ▪ keystore_hasKey() ▪ keystore_deleteKey() ▪ keystore_wipe() ◦ Usage Scenarios ▪ Generic Interaction between Application and CryptoServer ▪ Application imports a key ▪ Application creates a key	Sub Pages: • 2020-02-13 Meeting Notes Key Store V1 • 2020-02-17 Meeting Notes Key Store V1 - Follow Up • Full SEOS Key Store • SEOS KeyStore API v1.0 • SEOS KeyStore Core Library • SEOS KeyStore - Formally Verified Version • SEOS KeyStore usage • Terminology
---	---

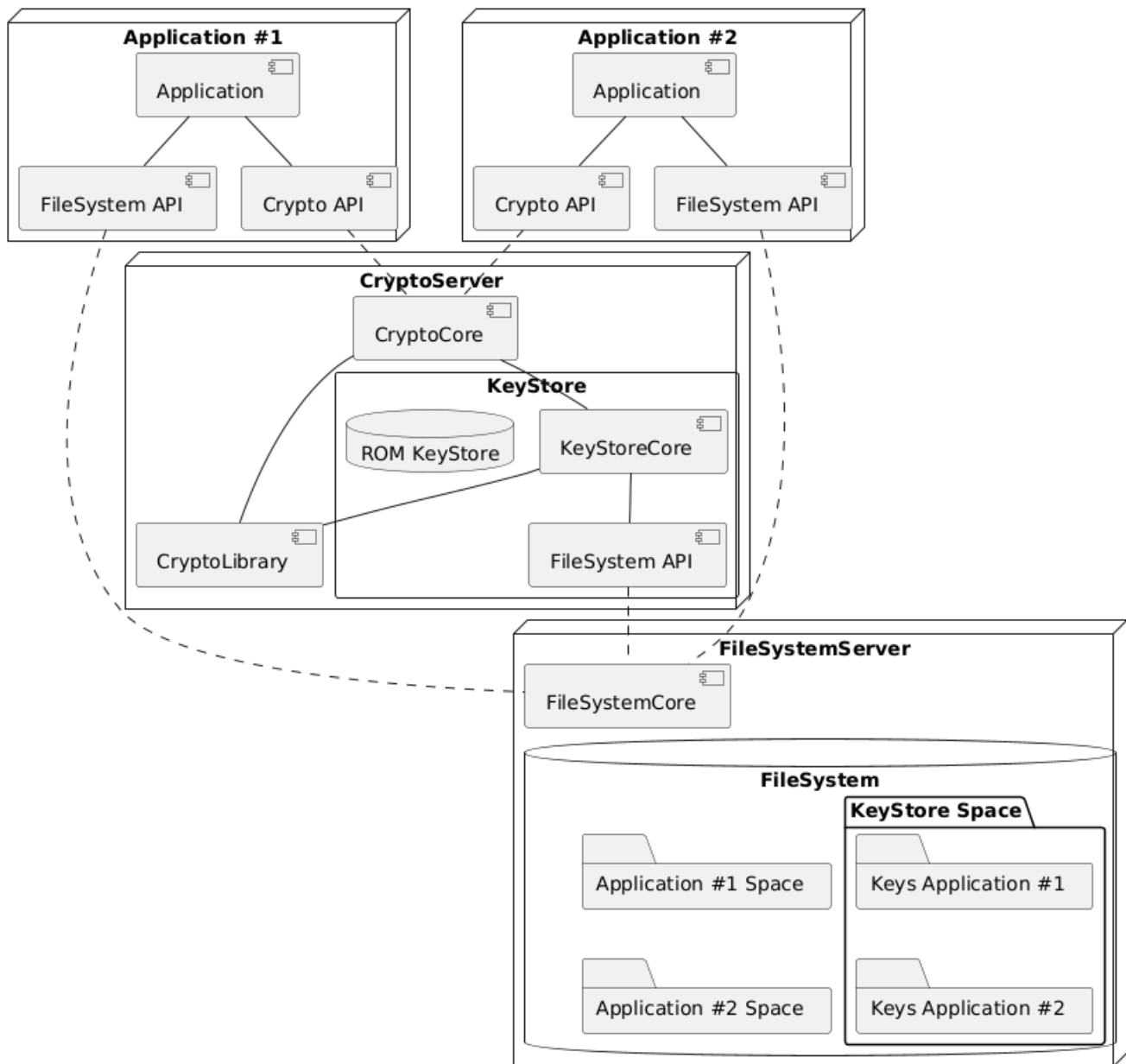
Overview

The key store is provided for applications, e.g. in conjunction with some key provision mechanism. This allows separating the application code from the application key management.

Generic requirements for a full-blown solution

- [keystore](#)
 - [store keys](#)
 - [support different access rights \(currently determined by calling application\)](#)
 - [future: support backup/restore](#)
 - [create key](#)
 - [delete key](#)
 - [import a key](#)
 - [export a key](#)
 - [use key](#)
 - [set key access rights](#)
- [internal functions](#)
 - [serialize a key to store it in non-volatile memory](#)
 - [deserialize a key from storage in non-volatile memory](#)
 - [future: backup/restore](#)

System Architecture



Minimal Keystore

Requirements based on input from Gernot, 2019-02-15

- **KeyStore**
 - provides a store partitioned by AppID, that associates raw key material with names.
 - does not provide key-specific meta data or knows details about what kind of key is being stored/retrieved. The raw key material can contain such meta-data, but is interpreted only by higher-level components (e.g. a crypto component).
 - does not provide access control or key delegation beyond partitioning by AppID, i.e. key sharing etc. are performed on a higher abstraction level.
 - Global keys (e.g. device key) are accessed indirectly through a designated trusted app (e.g. a designated crypto component). This means one designated partition stores global data and policy/access to that global data is managed explicitly by the component of that partition.
 - Key material is stored in ROM or obtained from a file system. Temporary caching key material in RAM might be an implementation optimization to minimize File I/O
- **File System**
 - is not trusted, thus any key material stored there must be encrypted with an internal key
 - each key is stored in a file, which can be accessed by a designated file name
 - the file content is independent of the file name or file system implementation, thus a key store key file can be moved and renamed
 - the file content contains meta data that identifies it as a specific key from an app. This meta data is not secret.

- a key file cannot impersonate another key file.
 - there is no replay protection in the file system, thus a file can be replaced with an older key blob of the same app_id/key_id. Replay protection can be implemented on higher layer, e.g. by checking a key's fingerprint with **key_fingerprint()**
- The system contains 2 global read-only key stores
 - system public key store is accessible by any application
 - system secret key store is restricted to system services

Threat Model/Assumptions

- a medium assurance (~ US secret) device with upgrade path to very high assurance (> CC EAL7)
- a physical tamper proof box, in particular physical tamper proof ROM for storing a secret device key and physically tamper protected RAM. Attempting to physically retrieve the device key or read RAM will destroy the key/RAM contents being accessed and stop the device.
- known, cryptographically signed code at boot time.
- high grade general confidentiality between apps and absence of timing channels are not a requirement
- apps/components on the device are of differing assurance and trust level, i.e. some apps are mutually untrusting, some are trusted, some are formally verified, some are un-analysed 3rd party drivers, etc.
- attackers can provide (any) malicious input (e.g. over a network)
- attackers might be able to subvert outward facing drivers and low-assurance components
- high (formally verified)/medium (analysed) assurance components have to be resistant to attack from malicious input (e.g. network) and subverted components
- the integrity of components (data and control flow) should not depend on unconnected/partitioned components.

Design Considerations

- KeyStore does not expose a direct interface for any task, but is part of the system's CryptoServer
- Applications interact with the CryptoServer, which will internally call the KeyStore then to obtain key material
- Rationale:
 - client/apps are of varying trust level and should therefore be treated like the outside network, i.e. should never see any raw (unencrypted) key material apart from public keys. The first level that has that kind of semantic key information is the crypto component. I.e. "export" operations must always go through a crypto component.
 - The above point means that delegation is necessary for at least key export. However, while clients connected directly to the key store can be identified by endpoint badge, delegating authority to do something with a key (e.g. to the crypto component) can easily become a complex protocol with additional threats (forging/guessing appIDs, since endpoint badges are local to the KS). The simple solution is to let the first layer with enough information handle client connections.
 - simplifies KS, because it can now trust AppIDs, since they are provided by a (trusted) crypto component.
 - key handles can be implemented in the crypto component at a lower assurance level without compromising critical key store properties.
- there are no functions to enumerate keys in a key store
 - it is assumed that the application has this information
 - a function is provided that erases the whole application specific key store and puts it in the state of an empty key store

Limitations

- system configuration must ensure exclusive accesses for the key store to the file system partition holding the key store data
- there is no replay-protection for the data in the file system
- keys persistent in the file system are small blobs of data, so all reads and writes are atomic.

State

- AppIDs:
 - static, small set of numbers (< ~2⁶).
 - These could be the badges client apps have with the crypto component that manages them.
 - If multiple crypto components are connected to the KS, this set needs to be (statically) partitioned by crypto the component.
 - The KS makes no assumptions on this, it just treats each AppID as a separate app, and trusts the connected crypto component to communicate the correct AppID for the application.
 - one dedicated AppID for global store
- key names: strings, if desired with name restriction, of small, static max length (< 2⁶).
- main state:
 - a table for each AppID from key name to key content.
 - Key content can be of varying size and is treated as binary blob.
 - The table is statically configured and pre-allocated to a maximum size.
 - Each AppID may have a ROM table of the same form; usually only the global table will have content.

Data structures

data_buffer_t

see [SEOS API#const_data_buffer_t](#)

const_data_buffer_t

see [SEOS API#data_buffer_t](#)

app_id_t

- an application ID is a non-modifiable unique identifier for an application on the device
- for the first implementation, application IDs will be a small integer value, e.g. between 0 and 64, i.e. the data buffer is just one byte

```
typedef const_data_buffer_t app_id_t;
```

key_id_t

- a key ID is a non-modifiable unique identifier for a key within an application.
- for the first implementation, key IDs will be a small integer value, e.g. between 0 and 64, i.e. the data buffer is just one byte

```
typedef const_data_buffer_t key_id_t;
```

Key Store Memory Management

- the key store implementation should support allocating a temporary buffer on the heap
- the first implementation may not support this feature, so all buffers must be provided by the caller.

keystore_internal_malloc()

```
void*
keystore_internal_malloc(
    size_t len
)
```

- allocates a block of memory and returns a pointer to it
- the content of the buffer will be filled with zeros
- implementation may have its own memory pool or just call the runtime library's `malloc()`

keystore_internal_free()

```
void
keystore_internal_free(
    void *ptr
)
```

- free a block of memory allocated from `keystore_internal_malloc()`
- the content of the buffer will be overwritten with zeros before releasing it.
- implementation may have its own memory pool or just call the runtime library's `free()`

Key Store Persistence System Connector

- the key store implementation uses a persistence system connector that must provide the API described below
- there is no explicit assumption that the persistence system is a full features file system
 - it may implement an application specific folder, where each key store exists
 - it may implement one file per application, where all keys are stored in
- The key store shall have a rough understanding about the file system to use the API in a proper and efficient way
- The interface is kept simple for the first implementation, as the key specific data is small
- The file system connector is responsible to map an `app_id` and `key_id` to a proper unique file identifier in the file system
 - for the first application, `app_id` and `key_id` are just a one byte integer each, thus an internal `getFileName()` function can return a 16-bit integer of the form `[app_id][key_id]`

keystore_fs_connector_putKeyContent()

```
returnCode_t
keystore_fs_connector_putKeyContent(
    app_id_t const      *app_id,
    key_id_t const      *key_id,
    const_data_buffer_t const *data
)
```

- functional description
 - puts a data blob into a key file
 - this function is suitable for small file content only, which is good enough for the first implementation of the key store
 - if the file does not exist, it is created
 - if the file exists, its content is overwritten
 - there might be meta-information in the file system, which must be updated also
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SOES_ACCESS_DENIED	file access denied
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_INSUFFICIENT_SPACE	storing this key exceeds key store capacity

- pseudo code implementations

```

keystore_fs_connector_putKeyContent(app_id, key_id, data)
{
    fileName = getFileName(app_id, key_id);
    hFile = file_openOrCreate(fileName);
    ret = file_write(hFile, data);
    file_close(hFile);
    return ret;
}

```

keystore_fs_connector_getKeyContent()

```

returnCode_t
keystore_fs_connector_getKeyContent(
    app_id_t const *app_id,
    key_id_t const *key_id,
    data_buffer_t *data
)

```

- functional description
 - get the file content of a key file
 - this function is suitable for small file content only, which is good enough for the first implementation of the key store
 - if **data.buffer** is not NULL then **data.len_max** contains the size of the buffer and **data.len_used** will be set to the size of the content
 - if **data.buffer** is NULL, then **data.len_max** must be:
 - 0 to indicate that the function is called for probing, on return **data.len_used** will be set to the size that is needed.
 - 1 to indicate the function should allocate the buffer internally and set **data.buffer**, **data.len_max** and **data.len_used** accordingly
 - any other value will make the function fail with SEOS_ERROR_INVALID_PARAMETER.
 - Passing **data.buffer = NULL** and **data.len_max = 0** is a perfectly valid operation to check if a file exists or to get the file content size in **data.len_used**.
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_NOT_FOUND	key file does not exist
SOES_ACCESS_DENIED	file access denied
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_NOT_SUPPORTED	operation not supported
SEOS_ERROR_BUFFER_TOO_SMALL	buffer too small, parameter fileName.len_used contains required length
SEOS_ERROR_OUT_OF_MEMORY	not enough memory to allocate buffers

- pseudo code implementations

```

keystore_fs_connector_getKeyContent(app_id, key_id, data)
{
    fileName = getFileName(app_id, key_id);
    hFile = file_openOrCreate(fileName);
    [ret,data] = file_read(hFile);
    file_close(hFile);
    return [ret,data]
}

```

keystore_fs_connector_deleteKey()

```

returnCode_t
keystore_fs_connector_deleteKey(
    app_id_t const *app_id,
    key_id_t const *key_id
)

```

- functional description
 - delete a key file
 - there might be meta-information in the file system, which must be updated also
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_NOT_FOUND	key does not exist in key store
SOES_ACCESS_DENIED	file access denied
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_NOT_SUPPORTED	operation not supported, e.g. for a key store that is not backed by a file system

- pseudo code implementations

```

keystore_fs_connector_deleteKey(app_id, key_id, data)
{
    fileName = getFileName(app_id, key_id);
    ret = file_delete(fileName);
    return ret;
}

```

keystore_fs_connector_wipe()

```

returnCode_t
keystore_fs_connector_wipe(
    app_id_t const *app_id,
)

```

- functional description
 - delete all key material and meta information for an **app_id** and effectively initialize an empty key store
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SOES_ACCESS_DENIED	file access denied
SOES_ERROR_OPERATION_FAILED	operation failed, key store may have been partially wiped.
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_NOT_SUPPORTED	operation not supported, e.g. for a key store that is not backed by a file system

- pseudo code implementations

```
keystore_fs_connector_wipe(app_id, key_id, data)
{
    bool_t isError = FALSE;
    foreach fileName in getFilesOfFolder(app_id)
    {
        ret = file_delete(fileName);
        if isError() { isError = TRUE; }
    }
    return isError ? SOES_ERROR_OPERATION_FAILED : SEOS_SUCCESS;
}
```

Key Store Crypto Helper Functions

- this is an internal API that may change in the future
- all buffer sizes can be hard-coded and a caller can be assumed to know that's required here
- in the first implementation, the key store is tightly bound to the CryptoServer
- the implementation can access the internals of the CryptoLibrary directly and not use the SOES API
- the implementation should avoid depending on too many internals of the CryptoLibrary implementation
- future implementation should use the [Crypto API](#) to allow separating the KeyStore from the CryptoServer

deriveBlobKey()

```
returnCode_t
deriveBlobKey(
    const_data_buffer_t *app_id,
    const_data_buffer_t *key_id,
    uint8_t *key[32]
)
```

- functional description
 - this is an internal helper function
 - create a unique key for a blob that is written to a file on an untrusted file system. The blob itself holds a serialized key object belonging to an application
 - the key does not depend on the underlying file system, thus the resulting file can be moved to another file system on the device and remains readable.
 - caller must provide the buffer of the derived key
 - key derivation shall use the standard key derivation function from the CryptoCore, e.g. PBKDF2 from [RFC8018](#)
 - blobKey = PBKDF2(HMAC-SHA256, KeyStoreRootKey, "SEOS-KeyStore-BlobSecret" || SHA-256(app_id) || SHA-256(key_id), 64, 256)
 - using SHA-256(app_id) and SHA-256(key_id) allows dealing with IDs of arbitrary length without the explicit need to store all lengths to prevent potential conflicts
 - KeyStoreRootKey is a device specific secret master key known by the key store implementation.
 - 64 rounds seem good enough for derivation purposes, maybe using 1 round would also be ok and we just use SHA-256() instead of PBKDF2()
 - blobKey is a 256 bit key can be used as AES-256 key.
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_NOT_SUPPORTED	operation not supported for the given parameters

keystore_internal_encrypt()

```
returnCode_t
keystore_internal_encrypt(
    app_id_t const *app_id,
    key_id_t const *key_id,
    const_data_buffer_t const *raw_key,
    uint8_t *ciphertext
)
```

- functional description
 - encrypt a raw key into a blob that is stored on the file system
 - for the first implementation, the caller knows the size of **plaintext** and the algorithm that is used, thus it must provide the proper buffer for **ciphertext**
 - algorithm is AEAD_AES_256_GCM (RFC5116, 32-byte key, 16 byte tag, random nonce as IV)
 - plaintext = raw_key
 - authentication data = "SEOS-KeyStore-Key" || SHA-256(app_id) || SHA-256(key_id)
 - using SHA-256(app_id) and SHA-256(key_id) allows dealing with IDs of arbitrary length without the explicit need to store the length to prevent potential conflicts
 - the AEAD tag will allow checking if the file content is valid, then reading the key
 - the random nonce ensures that the file content will vary each time, even if the key remains the same
 - it is unlikely that we ever run into the limits of AEAD_AES_256_GCM security
 - the encryption key is different for each **app_id** and **key_id**
 - the size of the raw key is usually less than 1 KiB and a few KiB at max.
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid

- pseudo code implementations

```
keystore_internal_encrypt(app_id, key_id, plaintext)
{
    blobKey = deriveBlobKey(app_id, key_id)
    auth_data = SHA-256( SHA-256(app_id) || SHA-256(key_id) )
    ciphertext = AEAD_AES_256_GCM_encrypt(blobKey, auth_data, plaintext)
    return ciphertext
}
```

keystore_internal_decrypt()

```
[raw_key] = keystore_internal_decrypt(
    app_id_t const      *app_id,
    key_id_t const      *key_id,
    const_data_buffer_t const *ciphertext
    uint8_t              *plaintext
)
```

- functional description
 - decrypt a blob that is stored on the file system to obtain the raw key
 - the **format** field defines how to encrypt the data.
 - Currently there is only the format 0x01 defined, which is described in **keystore_internal_encrypt()**.
 - unsupported formats must be discarded and result in the return code SEOS_ERROR_CORRUPTED_DATA
 - ciphertext** contains the actual encrypted data, then the tag
 - for the first implementation, the caller knows the size of **ciphertext** and the algorithm that is used, thus it must provide the proper buffer for **plaintext**
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_INVALID_PARAMETER	a parameter is invalid
SEOS_ERROR_CORRUPTED_DATA	decryption failed, e.g. due to invalid tag or unsupported format

pseudo code implementations

```

keystore_internal_decrypt(app_id, key_id, ciphertext)
{
    blobKey = deriveBlobKey(app_id, key_id)
    auth_data = SHA-256( SHA-256(app_id) || SHA-256(key_id) )
    [err_crypt,plaintext] = AEAD_AES_256_GCM_decrypt(blobKey, auth_data, ciphertext)
    ret = processErrorCode(fs_ret);
    return [ret, isOk(ret) ? plaintext : NULL]
}

```

KeyStore exposed API to CryptoServer

Concept

- this API is used by a trusted caller, which is the CryptoServer Core in the first implementation
- CryptoServer Core
 - can identify callers and determine their application ID
 - may cache keys obtained from a key store and use them in multiple operations.
 - knows which applications are allowed to access and manage other application's keys store.
 - every application has read-only access to the system's public key store
 - unless an application is a manager it must only be allowed to access its own key store
 - CryptoServer internal functions related to KeyStore usage
 - serializeKey(): serialize a key object into a buffer, called before calling **keystore_newKey()**
 - deserializeKey(): create a key object from a buffer containing a serialized key object provided by **keystore_getKey()**

API Properties

Basic store properties under the assumption that the key store has exclusive access to the underlying file system

- putting a key into a store
 - calling **keystore_getKey(app_id, key_id)** after a successful **keystore_newKey(app_id, key_id, raw_key)** returns **raw_key**
- overwriting a key in the store
 - calling **keystore_getKey(app_id, key_id)** after a successful **keystore_newKey(app_id, key_id, raw_key_2)** after a successful **keystore_newKey(app_id, key_id, raw_key_1)** returns **raw_key_2**
- deleting a key from the store
 - calling **keystore_getKey(app_id, key_id)** after a successful **keystore_deleteKey(app_id, key_id)** returns an error
 - calling **keystore_hasKey(app_id, key_id)** after a successful **keystore_deleteKey(app_id, key_id)** returns FALSE
 - calling **keystore_getKey(app_id, key_id)** after a successful **keystore_wipe(app_id)** returns an error
 - calling **keystore_hasKey(app_id, key_id)** after a successful **keystore_wipe(app_id)** returns FALSE
 - calling **keystore_deleteKey(app_id, key_id)** after a successful **keystore_deleteKey(app_id, key_id)** returns no error
 - calling **keystore_deleteKey(app_id, key_id)** after a successful **keystore_wipe(app_id)** returns no error
- partitioning
 - running any operation on **key_id_1** does not change the outcome of running any operation on a different **key_id_2**.
 - running any operation on **app_id_1** does not change the outcome of running any operation on a different **app_id_2**.
 - calling **keystore_getKey(app_id, key_id_1)** after a successful **keystore_deleteKey(app_id, key_id_2)** with **key_id_1** different from **key_id_2** returns the same as just running **keystore_getKey(app_id, key_id_1)**
 - calling **keystore_getKey(app_id, key_id_1)** after a successful **keystore_newKey(app_id, key_id_2, rawKey)** with **key_id_1** different from **key_id_2** returns the same as just running **keystore_getKey(app_id, key_id_1)**
- read operations don't change the key store state
 - calling **keystore_getKey(app_id, key_id)** does not change the state, i.e. **keystore_getKey(app_id, key_id)** then running any "f" is the same as just running "f"
 - calling **keystore_hasKey(app_id, key_id)** does not change the state, i.e. **keystore_hasKey(app_id, key_id)** then running any "f" is the same as just running "f"

keystore_newKey()

```

returnCode_t
keystore_newKey(
    app_id_t const      *app_id,
    key_id_t const      *key_id,
    const_data_buffer_t const *raw_key
)

```

- functional description
 - the function stores a new key in the application specific persistent key store
 - the caller must have verified that this key is allowed to be put into the key store of the given **app_id**
 - the persistent key store is implemented on top of an untrusted file system, thus the key data will be encrypted before actually persisting it

- **raw_key** is a serialized byte stream of a key object that exists in the CryptoServer core, the key store makes no assumptions about the details of the key and what internal format the byte stream uses
- the function may hide details about internal error when accessing the file system and just return generic error code to avoid leaking information.
- file content is

```
struct {
    struct {
        uint32_t format; // = 0x1
    } header
    uint8_t encrypted_raw_key_and_tag[];
}
```

- the **format** field leaves room for future extensions of the actual storage format.
- there is no need to store the length of the file **encrypted_raw_key_and_tag**, because this can be calculated from the file content length and the **format** field
- the first implementation knows that AEAD_AES_256_GCM is used and the buffer needed for **encrypted_raw_key_and_tag** is 16 byte longer than the raw key buffer

- return value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_NOT_SUPPORTED	operation not supported
SEOS_ERROR_ACCESS_DENIED	could not store key in key store

- implementation

```
keystore_newKey(app_id, key_id, raw_key)
{
    [ciphertext] = keystore_internal_encrypt(app_id, key_id, raw_key);
    fileContent = header || ciphertext;
    fs_ret = keystore_fs_connector_putFileContent(app_id, key_id, fileContent);
    ret = processErrorCode(fs_ret);
    return ret;
}
```

keystore_getKey()

```
returnCode_t
keystore_getKey(
    app_id_t const *app_id,
    key_id_t const *key_id,
    data_buffer_t *raw_key
)
```

- functional description
 - returns the raw key in **raw_key_buffer** for the given **app_id** and **key_id**
 - **len_raw_key_buffer** contains the buffer size on input and is set to the size that is actually used
 - **raw_key_buffer** may be used as a temporary buffer for internal operations
 - if **raw_key_buffer** is NULL, then **len_raw_key_buffer** will be set to the size that would be needed
 - the function may hide details about internal error when accessing the file system and just returns generic error code to avoid leaking information.
- returns value

Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_NOT_FOUND	key does not exist in key store
SEOS_ERROR_CORRUPTED_DATA	corrupted key data
SEOS_ERROR_BUFFER_TOO_SMALL	buffer too small, parameter len_raw_key_buffer contains required length

- pseudo code implementation

```

keystore_getKey(app_id, key_id)
{
    [fs_ret, fileContent] = keystore_fs_connector_getFileContent(app_id, key_id);
    ret = processErrorCode_fs(fs_ret)
    if !isOk(ret) { return [ret, NULL]; }

    [header, ciphertext] = processBlob(fileContent)
    if !checkHeader(header) { return [ret, NULL]; }
    [crypto_ret, raw_key] = keystore_internal_decrypt(app_id, key_id, ciphertext);
    ret = processErrorCode_fs(crypto_ret)
    if !isOk(ret) { return [ret, NULL]; }

    return [ret, raw_key];
}

```

keystore_hasKey()

```

bool_t
keystore_hasKey(
    app_id_t const *app_id,
    key_id_t const *key_id
)

```

- functional description
 - checks if a key exists in the keystore without loading it or performing any crypto operation
 - similar to **keystore_getKey()**, but returns just a boolean value instead of the raw key data
 - the function may hide details about internal error when accessing the file system and just returns generic error code to avoid leaking information.
- returns value

Error code	Description
TRUE	key exists in key store, however, there is no guarantee the key data can be retrieved
FALSE	key does not exist in key store

- pseudo code implementation

```

keystore_hasKey(app_id, key_id)
{
    [fs_ret, fileContent] = keystore_fs_connector_getFileContent(app_id, key_id);
    ret = processErrorCode_fs(fs_ret);
    return ret;
}

```

keystore_deleteKey()

```

returnCode_t
keystore_deleteKey(
    app_id_t const *app_id,
    key_id_t const *key_id
)

```

- functional description
 - removes the existing key `name` in the table of app_id.
 - the function may hide details about internal error when accessing the file system and just returns generic error code to avoid leaking information.
 - deletion of key may lead to fragmentation in the file system. This is acceptable for the first implementation, further details might also be handled in the file system connector.
- return value

Error code	Description
------------	-------------

SEOS_SUCCESS	operation successful
SEOS_ERROR_NOT_FOUND	file does not exist
SEOS_ERROR_ACCESS_DENIED	file access denied, e.g. due to read-only file system

- pseudo code implementation

```
keystore_deleteKey(app_id, key_id)
{
    fs_ret = keystore_fs_connector_deleteFile(app_id, key_id);
    ret = processErrorCode_fs(fs_ret);
    return ret;
}
```

keystore_wipe()

```
returnCode_t
keystore_wipe(
    app_id_t const *app_id
)
```

- functional description
 - removes all keys and meta data from an application specific key store
 - if the application key store does not exist or is empty, this function will not return an error
- return value

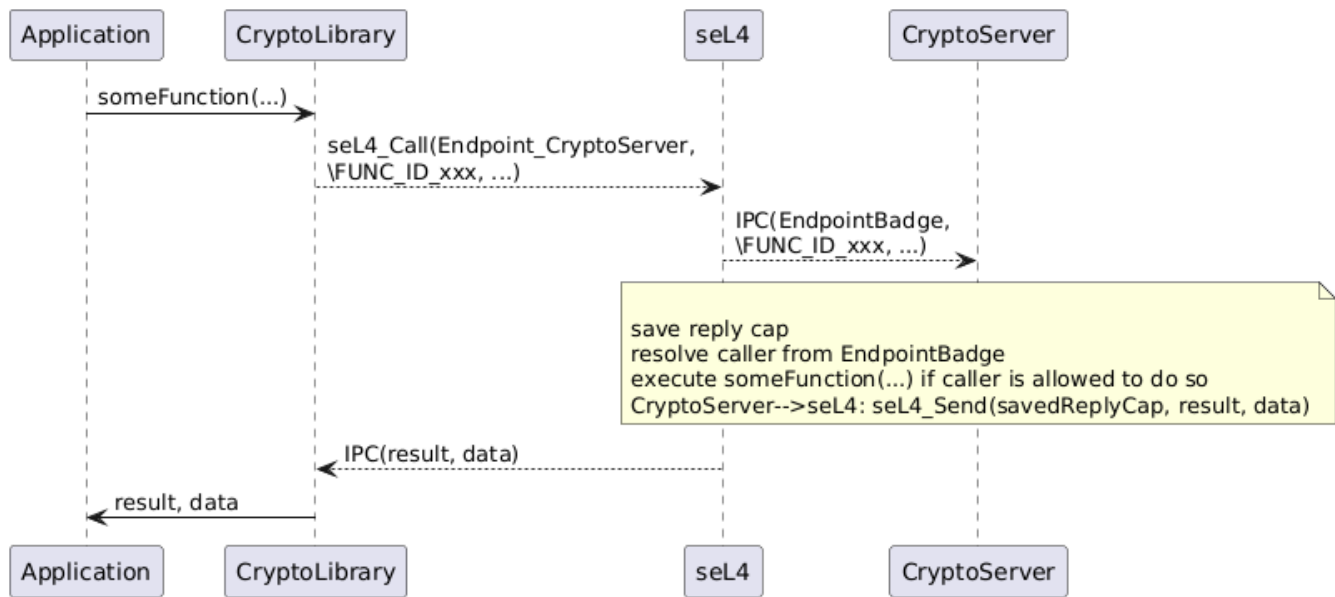
Error code	Description
SEOS_SUCCESS	operation successful
SEOS_ERROR_ACCESS_DENIED	file access denied, e.g. due to read-only file system

- pseudo code implementation

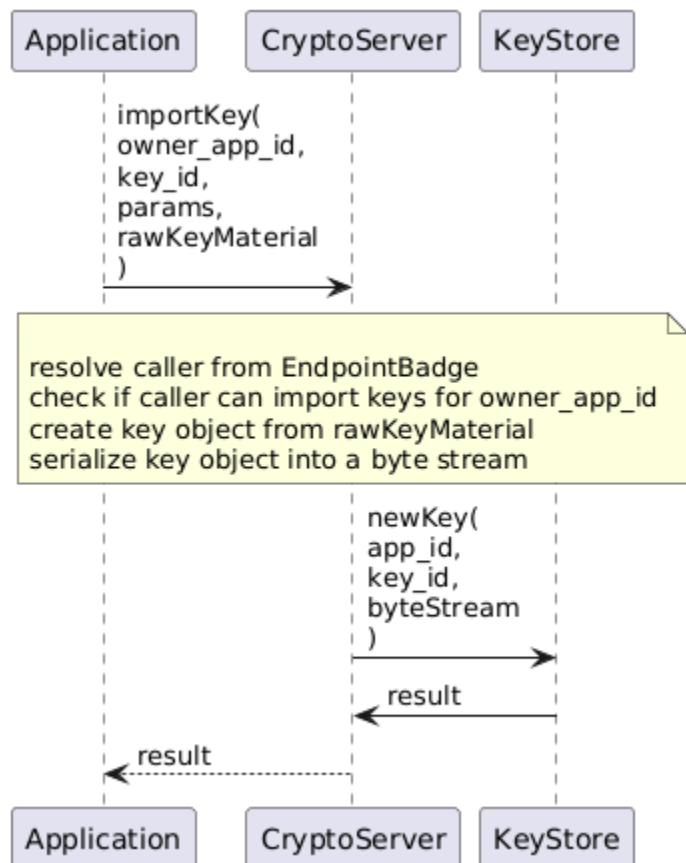
```
keystore_wipeAllKeys(app_id)
{
    fs_ret = keystore_fs_connector_wipe(app_id);
    ret = processErrorCode_fs(fs_ret);
    return ret;
}
```

Usage Scenarios

Generic Interaction between Application and CryptoServer



Application imports a key



Application creates a key

